L18: Routing and Probabilistic Branching

Simulation Without a Black Box

Outline

Learning Objectives

Network Diagram

Probabilistic Routing

Attribute-Based Routing

State-Dependent Routing

Join-Shortest-Queue vs. Random

Tandem Queue Example

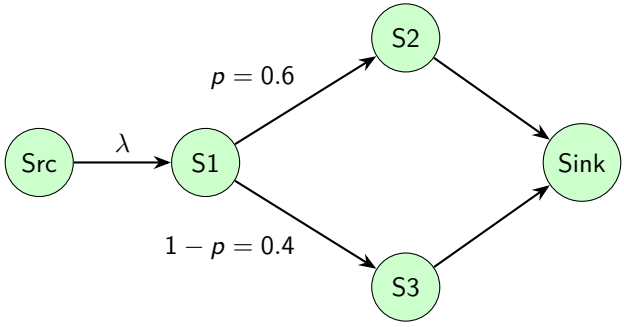
Key Takeaways

Learning Objectives

By the end of this lecture you will be able to:

- Implement probabilistic routing with `rng.random() < p`.
- Route entities based on customer attributes (class, acuity, type).
- Route state-dependently — send to shortest queue (JSQ).
- Draw and interpret a TikZ network diagram with routing probabilities.
- Build a tandem-queue model by passing entities between two SimPy processes.

Routing Network: TikZ Diagram



After stage S1, 60% of customers go to S2 (specialist), 40% to S3 (express lane). The downstream rates are $\lambda_2 = 0.6\lambda$ and $\lambda_3 = 0.4\lambda$.

Probabilistic Routing: `rng.random() < p`

```python
import simpy, numpy as np

def entity(env, s1, s2, s3, rng, waits):
    # Stage 1
    with s1.request() as req:
        yield req
        yield env.timeout(rng.exponential(1.0))

    # Probabilistic branch after stage 1
    if rng.random() < 0.6:
        next_server = s2      # specialist (60%)
    else:
        next_server = s3      # express (40%)

    # Stage 2 or 3
    with next_server.request() as req:
        arrive2 = env.now
        yield req
        waits.append(env.now - arrive2)
        yield env.timeout(rng.exponential(0.8))
```

Attribute-Based Routing: Route on Customer Type

```python
from dataclasses import dataclass

@dataclass
class Patient:
    acuity: int      # 1=critical, 2=urgent, 3=routine
    arrive: float

def route_patient(patient, servers):
    """Route to the appropriate care team."""
    if patient.acuity == 1:
        return servers["resus"]
    elif patient.acuity == 2:
        return servers["fast_track"]
    else:
        return servers["waiting_room"]

def patient_process(env, patient, servers, rng, waits):
    server = route_patient(patient, servers)
    with server.request() as req:
        yield req
        waits[patient.acuity].append(env.now - patient.arrive)
        yield env.timeout(rng.exponential(1.0))
```

State-Dependent Routing: Join-Shortest-Queue (JSQ)

```
def jsq(servers):
    """Return the server with the shortest current queue."""
    return min(servers,
               key=lambda s: s.count + len(s.queue))

def entity(env, servers, rng, waits):
    arrive = env.now
    server = jsq(servers)          # re-evaluate at arrival time
    with server.request() as req:
        yield req
        waits.append(env.now - arrive)
        yield env.timeout(rng.exponential(1.0))
```

Why JSQ matters

JSQ achieves near-optimal performance with c servers: it is typically within a factor of 2 of the M/M/c bound. It requires *observable* queue lengths at each server.

- `s.count`: number currently in service.
- `len(s.queue)`: number waiting.

JSQ vs. Random Routing: Comparison

Setup: 2 servers, $\lambda = 1.6$, $\mu = 1.0$ per server, $\rho = 0.8$.

Policy	W_q (sim)	Std. error
M/M/2 (optimal)	1.78	0.04
JSQ (2 servers)	1.82	0.04
Random routing	4.00	0.09
Dedicated (worst)	4.00	—

JSQ nearly matches M/M/2; random routing is as bad as two dedicated M/M/1 queues.

Caveat

JSQ requires customers to observe queue lengths. If queues are hidden (phone systems, invisible back-room servers), use random routing or least-loaded routing with estimated load.

Tandem Queue: Chaining Generators

```
def entity(env, s1, s2, rng, waits1, waits2):
    """Two-stage tandem: S1 then S2."""
    # Stage 1
    a1 = env.now
    with s1.request() as req:
        yield req
        waits1.append(env.now - a1)
        yield env.timeout(rng.exponential(1.0))

    # Stage 2 (output of S1 is input to S2)
    a2 = env.now
    with s2.request() as req:
        yield req
        waits2.append(env.now - a2)
        yield env.timeout(rng.exponential(0.8))
```

Burke's theorem (preview)

If S1 is M/M/c, its departure process is Poisson with rate λ. So S2 also has Poisson input — the tandem has a product-form solution. This is L20.

Key Takeaways

Core ideas from L18

1. **Probabilistic routing**: `rng.random() < p` — always use the simulation RNG, not `random.random()`.
2. **Attribute routing**: attach a type field to the entity; route in a function.
3. **JSQ** nearly matches M/M/c and dramatically outperforms random routing — use it whenever queues are observable.
4. **Tandem queues**: chain generators; pass the entity implicitly through generator scope.
5. Draw the routing network first; assign $\lambda_k = p_k \lambda$ to each downstream stage; check $\rho_k < 1$.

Next: L19 — SimPy Store and Container: heterogeneous items and bulk resources.